

WEEK PRACTICAL – 01

```
saicharan_reddi@SAICHARANREDDY:~# id
1.id: uid=0(saicharan_reddi) gid=0(saicharan_reddi) groups=0(saicharan_reddi)
```

Report: After running the id command, I could see my user details. It showed my username as **Charan**, along with the user ID, group ID, and the groups I belong to like sudo and plugdev.

2.env:

```
_DISTRO_NAME=Ubuntu
ME=SAICHARANREDDY
D=/root
GNAME=root
TD_SHOWN=update-motd
ME=/root
NG=C.UTF-8
_INTEROPP=/run/WSL/392_interop
_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01:35:do=01;35:bd=40;33:0
:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;31:*.taz=01;31:*.lh
t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=01
:l=31:*.tbz2=01;31:*.tz=01;31:*.deb=01;31:*.rpm=01;31:*.jar=01;31:*.war=01
01;31:*.7z=01;31:*.xz=01;31:*.cab=01;31:*.wim=01;31:*.swm=01;31:*.dwm=01;
if=01;35:*.bmp=01;35:*.pbm=01;35:*.pgm=01;35:*.ppm=01;35:*.tga=01;35:*.xbm
:mng=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;35:*.mpeg=01;35:*.m2v=01;35:*.
35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01;35:*.rm=01;35:*.
*:xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgy=01;35:*.emf=01;35:*.ogv=01;35:*.
*:mka=00;36:*.mp3=00;36:*.mpc=00;36:*.ogg=00;36:*.ra=00;36:*.wav=00;36:*.
download=00;90:*.dpkg-dist=00;90:*.dpkg-new=00;90:*.dpkg-old=00;90:*.dpkg-
ig=00;90:*.rpmsave=00;90:*.swp=00;90:*.tmp=00;90:*.ucf-dist=00;90:*.ucf-ne
YLAND_DISPLAY=wayland-0
SSCLOSE=/usr/bin/lesspipe %s %s
RM=xterm-256color
SSOPEN=| /usr/bin/lesspipe %s
ER=root
SPLAY=:0
LVL=1
G_RUNTIME_DIR=/run/user/0
LENV=
G_DATA_DIRS=/usr/local/share:/usr/share:/var/lib/snapd/desktop
TH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games
avapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt
c:/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/windows
t/c/Program Files/nodejs:/mnt/c/Users/saicharan_6wg39hp/AppData/Local/Microsc
/bin:/mnt/c/Program Files/JetBrains/PyCharm 2026.1/bin:/mnt/c/Users/saichar
_reddi/AppData/Local/OfIce/12.1.0.274586/clitool:/snap/bin
US_SESSION_BUS_ADDRESS=unix:path=/run/user/0/bus
STTYPE=x86_64
LSE_SERVER=unix:/mnt/wslg/PulseServer
/usr/bin/printenv
saicharan_reddi@SAICHARANREDDY:~#
```

Report: When I executed the env command, many environment variables were displayed. I observed variables like HOME=/home/charan, USER=charan, PWD=/home/charan/OSSP, and PATH, which contain information about the system environment.

3.Export:

```
35:*.*m4v=01;35:*.*mp4v=01;35:*.*vob=01;35:*.*qt=01;35:*.*nuv=01;35:*.*wmv=01;35:*.*asf=01;35:*.*rm=01;35:*.*rmvb=01;35:*.*flc=01;35:*.*avi=01;35:*.*fli=01;35:*.*flv=01;35:*.*gl=01;35:*.*dl=01;35:*.*xcf=01;35:*.*xwd=01;35:*.*yuv=01;35:*.*cgm=01;35:*.*emf=01;35:*.*ogv=01;35:*.*ogx=01;35:*.*aac=00;36:*.*au=00;36:*.*flac=00;36:*.*m4a=00;36:*.*mid=00;36:*.*midi=00;36:*.*mka=00;36:*.*mp3=00;36:*.*mpc=00;36:*.*ogg=00;36:*.*ra=00;36:*.*wav=00;36:*.*oga=00;36:*.*opus=00;36:*.*spx=00;36:*.*xspf=00;36:*.*~=00;90:*.*#=00;90:*.*bak=00;90:*.*crdownload=00;90:*.*dpkg-dist=00;90:*.*dpkg-new=00;90:*.*dpkg-old=00;90:*.*dpkg-tmp=00;90:*.*old=00;90:*.*orig=00;90:*.*part=00;90:*.*rej=00;90:*.*rpmnew=00;90:*.*rpmorig=00;90:*.*rpmsave=00;90:*.*swp=00;90:*.*tmp=00;90:*.*ucf-dist=00;90:*.*ucf-new=00;90:*.*ucf-old=00;90:}"
declare -x NAME="SRIVALLI"
declare -x OLDPWD
declare -x PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/Program Files/dotnet:/mnt/c/Program Files/nodejs:/mnt/c/Users/sriva_6wg39hp/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/sriva_6wg39hp/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Program Files/JetBrains/PyCharm 2026.1/bin:/mnt/c/Users/sriva_6wg39hp/AppData/Roaming/npm:/mnt/c/Users/sriva_6wg39hp/AppData/Local/Kingsoft/WPS Office/12.1.0.27458/clitool:/snap/bin"
declare -x PULSE_SERVER="unix:/mnt/wslg/PulseServer"
declare -x PWD="/root"
declare -x SHELL="/bin/bash"
declare -x SHLVL="1"
declare -x TERM="xterm-256color"
declare -x USER="root"
declare -x WAYLAND_DISPLAY="wayland-0"
declare -x WSL2_GUI_APPS_ENABLED="1"
declare -x WSLENV=""
declare -x WSL_DISTRO_NAME="Ubuntu"
declare -x WSL_INTEROP="/run/WSL/643_interop"
declare -x XDG_DATA_DIRS="/usr/local/share:/usr/share:/var/lib/snapd/desktop"
declare -x XDG_RUNTIME_DIR="/run/user/0"
```

Report: The export command displayed the exported environment variables. I observed that variables such as HOME, USER, PWD, PATH, and other system variables were available and exported for the current session

4.Alias:

```
saicharan_reddi@SAICHARANREDDY:~$ alias
alias
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l='ls -CF'
alias la='ls -A'
alias ll='ls -alF'
alias ls='ls --color=auto'
```

Report: After executing the alias command, the terminal displayed a list of shortcut commands. I observed aliases like grep, ls, and ll, which are used as shortcuts for commonly used commands

TASK-1 Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to 3 another. Explain how control transitions between user Space and kernel space during execution

```
saicharan_reddi@SAICHARAN  ×  +  ▾
GNU nano 8.7.1                                copyfile.c *
#include <unistd.h>
#include <stdlib.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[])
{
    int source, destination;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    source = open(argv[1], O_RDONLY);

    if (source < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0)
    {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    while ((bytes_read = read(source, buffer, BUFFER_SIZE)) > 0)
```

```
saicharan_reddi@SAICHARANREDDY:~$ nano filecopy.c
saicharan_reddi@SAICHARANREDDY:~$ gcc -Wall -Wextra filecopy.c -o bin/filecopy
saicharan_reddi@SAICHARANREDDY:~$ echo "This is a sample file for system call testing." > sample.txt
saicharan_reddi@SAICHARANREDDY:~$ ./bin/filecopy sample.txt copy.txt
File copied successfully.

saicharan_reddi@SAICHARANREDDY:~$ cat sample.txt
This is a sample file for system call testing.

saicharan_reddi@SAICHARANREDDY:~$ cat copy.txt
This is a sample file for system call testing.
```

Report: I developed a C program to copy the contents of one file to another using the system calls `open()`, `read()`, `write()`, and `close()`. The source file was opened in read-only mode and the destination file was created for writing. The contents were read into a buffer and written to the destination file. During execution, the system calls transfer control from user space to kernel space, where the kernel performs the required file operations and returns the results to the program. The file was copied successfully.

Task-2 : Use the strace utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system 3 calls and identify the kernel services involved

```

saicharan_reddi@SAICHARANREDDY:~$ strace -e trace=openat,read,write,close cat sample.txt
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libcpre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
read(3, "nodev\ttsysfs\nnodev\ttmpfs\nnodev\tbd"..., 1024) = 442
close(3) = 0
openat(AT_FDCWD, "/proc/mounts", O_RDONLY|O_CLOEXEC) = 3
read(3, "none /usr/lib/modules/6.18.33.2-"..., 1024) = 1024
read(3, "ime 0 0\ndevpts /dev/pts devpts r"..., 1024) = 1024
read(3, "nosuid,nodev,noexec,relatime,no"..., 1024) = 1024
read(3, ".snapfuse ro,nodev,relatime,user"..., 1024) = 479
read(3, "", 1024) = 0
close(3) = 0
openat(AT_FDCWD, "/proc/self/maps", O_RDONLY|O_CLOEXEC) = 3
read(3, "5ea28c15c000-5ea28c24e000 r--p 0"..., 1024) = 1024
read(3, "adfe000-72b9adff000 r--p 000a90"..., 1024) = 1024
read(3, "00 08:30 13639"..., 1024) = 1024
read(3, " /usr/lib/x86_64-lin"..., 1024) = 1024
read(3, "0 13633 /us"..., 1024) = 566
close(3) = 0
openat(AT_FDCWD, "/usr/share/coreutils/locales/ucore/en-US.ftl", O_RDONLY|O_CLOEXEC) = 3
read(3, "# Common strings shared across a"..., 4080) = 4080
read(3, "", 32) = 0
close(3) = 0
openat(AT_FDCWD, "/usr/share/coreutils/locales/cat/en-US.ftl", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
write(2, "cat", 3cat) = 3

write(2, "cat", 3cat) = 3
write(2, ": ", 2:) = 2
write(2, "sample.txt: No such file or dire"..., 37sample.txt: No such file or directory) = 37
write(2, "\n", 1) = 1
+++ exited with 1 +++
saicharan_reddi@SAICHARANREDDY:~$

```

Report: I used the strace utility to trace the system calls generated by the command `cat sample.txt`. The trace showed important system calls such as `openat()`, `read()`, `write()`, and `close()`. `openat()` opens the file, `read()` obtains the file contents, `write()` displays the contents on the terminal, and `close()` closes the file descriptor. The experiment helped me understand how a user-level command interacts with kernel services through system calls.